

Document Object Model

학습 체크리스트 (1/2)

- ☐ HTML 문서가 브라우저에 의해 객체 트리 구조인 DOM으로 파싱되는 원리 이해
- ☐ DOM 트리 내에서 노드(Node)와 엘리먼트(Element)의 상속 관계 및 기능 차이 구분
- ☐ querySelector 및 querySelectorAll을 활용해 CSS 선택자 기반으로 요소를 탐색하는 방법 숙지
- ☐ createElement, textContent, append/prepend 등을 사용해 안전한 DOM 동적 생성 기법 구현

학습 체크리스트 (2/2)

- ☐ classList API, setAttribute, dataset 등을 이용해 안전하게 요소를 변경하는 최신 기법 활용
- ☐ 부모 노드를 거치지 않고 대상 엘리먼트를 직접 제거하는 `element.remove()` 방식 적용
- ☐ innerHTML 사용 시 발생할 수 있는 XSS 보안 취약성 인지 및 `textContent` 대체 수단 활용

DOM이란?

문서 객체 모델 (Document Object Model)

브라우저가 HTML 문서를 파싱하여 생성하는 객체 트리 구조의 프로그래밍 인터페이스

- **동적 제어 허용:** JavaScript를 사용해 웹 페이지의 구조, 스타일, 내용을 동적으로 탐색 및 제어 가능
- **DOM 트리 구조:** HTML의 계층적 구조를 객체 트리 노드로 정밀하게 매핑하여 메모리에 탑재
- **기본 시작점:** 최상위 `document` 객체를 출발지로 삼아 하위 요소로 계층이 파고드는 구조

Node vs Element 상속 관계

노드 (Node)

DOM 트리를 구성하는 가장 기본적인 추상 구성 단위이자 상위 인터페이스

- **상속 아키텍처:** 모든 DOM 객체는 기본 `Node` 인터페이스를 부모로 상속받아 파생됨
- **노드의 종류:** 요소(Element), 텍스트(Text), 주석(Comment), 문서(Document) 등 트리 구성원 전체 포함
- **엘리먼트의 정의:** `Node` 를 상속받은 하위 클래스이며, 오직 실제 HTML 마크업 태그(요소)만을 지칭
- **전용 조작 기능:** 엘리먼트는 어트리뷰트, 클래스명, 스타일 등을 변경할 수 있는 풍부한 제어 도구 제공

DOM 탐색 querySelector

쿼리 셀렉터 (querySelector)

CSS 선택자 문법을 활용하여 DOM 트리 내에서 원하는 단일 또는 다중 요소를 탐색하는 현대 표준 API

- 단일 요소 선택 (`querySelector`): CSS 선택자와 매칭되는 최초의 단일 엘리먼트 객체 반환 (없으면 `null`)
- 다중 요소 선택 (`querySelectorAll`): 매칭되는 모든 요소를 담은 정적 노드 리스트(`NodeList`) 반환
- 예측 가능성: 상태 변화가 실시간 반영되어 오작동을 초래하던 구식 `HTMLCollection` 을 완전히 대체

querySelector 다중 요소 제어

- 설명: `querySelectorAll` 이 반환하는 `NodeList` 를 효과적으로 활용하고 다루는 방법입니다.
- 주요 활용 규칙:
 - 자체 반복문 지원: `NodeList` 는 배열이 아니지만, 자체 `forEach` 메서드를 직접 실행하여 순회 가능
 - 진짜 배열 형변환: 고차 함수(`filter`, `map` 등)를 활용하려면 `Array.from()` 을 통한 변환 필수
 - 성능 및 안전성: 정적 스냅샷 형태의 목록을 보장하므로 예측하기 쉬운 안전한 제어 실현

CSS 선택자 기반 단일 및 다중 요소 탐색

```
1 // 1. CSS 선택자로 단일 요소 선택
2 const activeItem = document.querySelector('.list-item.active');
3
4 // 2. 다중 요소 선택 후 순회 처리
5 const cards = document.querySelectorAll('.card');
6 cards.forEach(card => card.style.borderColor = 'blue');
7
8 // 3. 고차 함수 활용을 위해 NodeList를 진짜 배열로 형변환
9 const activeCards = Array.from(cards)
10    .filter(card => card.classList.contains('active'));
```


노드 탐색 vs 엘리먼트 탐색

- 설명: DOM 트리 내에서 부모, 자식, 형제 방향으로 인접 요소를 탐색할 때의 두 가지 경로입니다.
- 경로 비교:
 - 엘리먼트 탐색 (강력 권장): 줄바꿈, 공백 등 보이지 않는 텍스트 노드를 제외하고 실제 태그 단위로 탐색
 - API: `parentElement`, `children`, `firstElementChild`, `nextElementSibling` 등
 - 노드 탐색 (주의): 공백이나 주석 노드를 탐색 범위에 포함하여 예기치 못한 탐색 실패 가능
 - API: `parentNode`, `childNodes`, `firstChild`, `nextSibling` 등

DOM 동적 생성 및 현대적 삽입

요소 동적 생성 (createElement)

브라우저 메모리상에 명시한 태그명을 가진 빈 엘리먼트 객체를 즉시 생성하여 대기시키는 API

- 현대적 삽입 API (`append` / `prepend`): 여러 개의 노드 객체와 일반 문자열을 동시에 한 번에 삽입 가능
- 레거시 방식 (`appendChild`) 대비 장점:
 - `appendChild` 는 문자열 삽입이 불가능하며, 오직 단 하나의 노드 객체만 인자로 전달 가능
 - `append` 는 요소 자식 목록 맨 뒤에, `prepend` 는 요소 자식 목록 맨 앞에 정밀 배치

메모리상 요소 생성 및 다중 자식/문자열 삽입

```
1 // 1. 동적 요소 생성 및 속성 정의
2 const newDiv = document.createElement('div');
3 newDiv.textContent = '동적 생성 div';
4 newDiv.classList.add('box', 'dynamic');
5
6 // 2. 현대적 삽입 API 활용 (노드와 문자열을 동시에 한 번에 추가)
7 const container = document.querySelector('.container');
8 container.append(newDiv, "텍스트 내용 바로 추가");
```

정밀 위치 삽입 insertAdjacentElement

정밀 위치 삽입 (insertAdjacentElement)

지정한 타겟 엘리먼트를 기준으로 정밀하게 구분된 네 가지 상대 위치에 새 요소를 삽입하는 API

- `'beforebegin'` : 타겟 요소 바로 앞 (형제 노드로 추가)
- `'afterbegin'` : 타겟 요소 내부의 첫 번째 자식으로 추가
- `'beforeend'` : 타겟 요소 내부의 마지막 자식으로 추가
- `'afterend'` : 타겟 요소 바로 뒤 (형제 노드로 추가)

지정된 정밀 위치 기준 동적 요소 삽입

```
1  const target = document.querySelector('.target');
2  const alertSpan = document.createElement('span');
3  alertSpan.textContent = '[공지] ';
4
5  // target 엘리먼트 바로 앞(beforebegin) 위치에 형제 노드로 삽입
6  target.insertAdjacentElement('beforebegin', alertSpan);
```

안전한 텍스트 및 HTML 수정

- **설명:** 웹 보안 표준을 위반하지 않으면서 엘리먼트 내부의 콘텐츠를 변경하는 기술입니다.
- **수정 API 비교:**
 - `textContent` (**강력 권장**): 주입된 문자열을 단순 글자로만 취급하며 HTML 해석을 원천 차단
 - **보안성:** 악성 스크립트 코드가 실행되지 않고 이스케이프되므로 XSS 공격에 완벽 대비
 - `innerHTML` (**주의 요망**): 주입된 문자열을 HTML 마크업으로 파싱하여 주입
 - **취약성:** 검증되지 않은 외부 사용자의 임의 입력을 그대로 전달할 시 스크립트 실행 취약점 초래

크로스 사이트 스크립팅(XSS) 원리

크로스 사이트 스크립팅 (Cross-Site Scripting)

웹 브라우저 상에서 신뢰할 수 없는 악성 스크립트를 동적으로 주입 및 실행시켜 세션을 탈취하거나 민감 정보를 가로채는 보안 취약점

- DOM 기반 XSS: 사용자의 폼 입력, URL 해시(`location.hash`) 등에 삽입된 악성 코드가 원인
- 보안 새니타이징 누락: 악성 문자열이 JS 필터링을 거치지 않고 `innerHTML` 등의 취약한 API에 직접 기입됨
- 메모리 강제 구동: 브라우저가 이 위험한 텍스트를 실제 스크립트로 판단하여 메모리 상에서 실행

XSS 위협 상황과 방어 대책

- 위협 상황:

- 세션/쿠키 하이재킹: `document.cookie` 에 은밀히 접근하여 로그인 토큰을 공격자 서버로 유출
- 피싱 UI 생성: 가짜 로그인 화면을 동적으로 노출시켜 사용자의 자격 증명(ID/PW) 무단 수집
- 강제 리다이렉트: `location.href` 조작을 통해 악성코드 설치나 스팸 사이트로 연결

- 방어 대책 (Best Practices):

- `textContent` 필수 사용: HTML 태그를 해석할 필요가 없는 단순 문자열은 무조건 `textContent`로 기입
- `DOMPurify` 도입: 불가피한 HTML 코드 주입 시, 악성 코드 및 이벤트 핸들러를 검증하는 라이브러리 가동
- CSP 규정 수립: 웹 헤더에 콘텐츠 보안 정책을 설정하여 승인되지 않은 인라인 스크립트 작동 완전 규제

textContent 기반 안전한 텍스트 주입

```
1  const desc = document.querySelector('.desc');
2
3  // 1. 안전한 텍스트 주입 (strong 태그가 해석되지 않고 문자로 단순 노출)
4  desc.textContent = '<strong>안전 확인</strong>';
5
6  // 2. 마크업 삽입 (오직 신뢰하는 내부 정적 코드에 한해서만 제한적 활용)
7  desc.innerHTML = '<em>안전한 내부 로컬 마크업</em>';
```

클래스 목록 및 사용자 정의 데이터 제어

- 설명: 최신 표준 명세에 부합하는 안전한 엘리먼트 속성 및 데이터 조작 기술입니다.
- 제어 API 구성:
 - 클래스 조작 (`classList`): add, remove, toggle, contains를 통해 안전하고 직관적인 조작 구현
 - 데이터셋 조작 (`dataset`): HTML5 `data-` 속성을 활용해 JS의 CamelCase 속성 형태로 데이터 공유
 - 인라인 스타일 (`style`): CSS 속성명을 JS 식별자 규격인 CamelCase로 자동 변환하여 직접 제어

클래스 목록 조작 및 사용자 속성/스타일 제어

```
1  const el = document.querySelector('.box');
2
3  // 1. classList API를 이용한 클래스 제어
4  el.classList.add('active');
5  el.classList.toggle('visible');
6
7  // 2. data- 속성을 dataset API로 조작 (data-user-role에 대응)
8  el.dataset.userRole = 'manager';
9
10 // 3. 인라인 스타일을 CamelCase 규칙으로 변환 적용
11 el.style.backgroundColor = '#eaeaea';
```

DOM 삭제 API의 현대화

- 설명: 필요 없어진 요소를 메모리와 화면에서 안전하게 수거하고 파괴하는 방법입니다.
- 두 API 비교:
 - 현대적 삭제 API (`remove()`): 대상 엘리먼트 스스로 자신을 직접 DOM 트리에서 영구 제거
 - 장점: 부모 노드를 굳이 거칠 필요가 없어 불필요한 역방향 참조가 사라지고 코드 가독성 극대화
 - 구식 삭제 API (`removeChild()`): 반드시 부모 노드 객체를 참조하여 지정된 자식을 제거하는 레거시 메서드
 - 용도: 레거시 환경(IE 계열 등 구형 브라우저)에 대한 철저한 하위 호환성이 요구될 때 적용

부모 노드 탐색 없는 현대적 요소 삭제

```
1 // 1. 현대적 삭제 API: 자기 자신을 트리에서 직접 제거
2 const ad = document.querySelector('.banner-ad');
3 if (ad) ad.remove();
4
5 // 2. 레거시 호환 API: 부모 엘리먼트를 거쳐서 자식 요소 제거
6 const list = document.querySelector('#item-list');
7 const item = document.querySelector('#item-first');
8 list.removeChild(item);
```